コードの読み方

9行目

```
public String index() {
```

public は、このメソッドが外部からアクセス可能であることを意味します。String は、メソッドが文字列を返すことを示しています。index() はメソッド名で、このメソッドが何をするのかを表します。**Controller の中のメソッドは、通常は戻り値をString にします。**

10行目

```
return "index";
```

return には文字列を指定します。通常、この文字列は表示させたいhtml ファイル名を指定します。

コードの読み方

2行目

```
<html xmlns:th="http://www.thymeleaf.org">
```

このビューの特徴として、2 行目に注目してください。ここでは、Spring Boot と相性が良いテンプレートエンジン「Thymeleaf」を使用するという宣言をしています。
xmlns:th を記載することで、HTML ファイル内でThymeleaf の機能が利用可能になります。

10行目

```
<div>トップページ</div>
```

[トップページ]と表示される部分（10 行目）からは、ユーザに見える画面の内容が入ります。今後、さまざまな情報やデザイン要素を追加していくことになります。

コードの読み方

10行目

```
<form th:action = "@{/login}" method="POST">
```

〈form〉要素を用いてデータ送信のためのフォームを作成します。この要素のth:action 属性は、フォームデータが送信されるサーバー上のURL を指定できます。Thymeleaf では、ただのaction ではなく、**th:action** を使うことで、**実行環境に変化があってもURL を正しく解決**できます。**「@」は、アプリケーションのコンテキストパス（基本URL）を、自動生成**してくれます。URLが異なるサーバーで実行しても、影響がなくなる効果があります。
ここでは/login がそのURL として指定されています。通常、**th:action 属性のURL 先は、@Controller のメソッドを指定**します。そして、method という属性がありますが、この属性は一般的なGET 方式の送信ではなく、POST 方式の送信をするという意味です。POST 方式にすると、パスワードがURL に表示されないなどのメリットがあり、隠したいデータを送信する時によく使われます。

コードの読み方

11行目・12行目

```
<p>社員番号　：<input type = "text" name = "id"/></p>
<p>パスワード：<input type = "password" name = "password"/></p>
```

<input>要素はユーザがデータを入力するためのフィールドを提供します。type="text"を指定することでテキスト入力フィールドが、type="password"を指定することでパスワード入力フィールドがそれぞれ生成されます。ここで、このフォームのデータは、社員番号という日本語で認識されるのではなく、**name 属性の値で認識されます。今回は、「id」と「password」というnameでデータが送られるということを覚えてください。**

13行目

```
<input type = "submit" value = "ログイン">
```

<input type="submit">は送信ボタンを表します。このボタンをクリックすると、フォーム内のデータがth:action 属性で指定されたURL に送信されます。value 属性はボタン上に表示されるテキストを定義します。

コードの読み方

10行目

```
@PostMapping("/login")
```

@PostMapping は、POST リクエストが送られた際に、このメソッドが実行されることを示しています。3 行でも同じ/login というURL を待ち受けていますが、普通のGet 方式での呼び出しがあった場合は、showLoginView()メソッドが実行され、Post 方式の呼び出しがあった場合は、こちらのdoLogin メソッドが実行されます。

11行目

```
public String doLogin(int id, String password) {
```

login_input.html から送信したデータのname 属性に対応する値を受け取るために使用されています。**Spring では、Controller のメソッドの仮引数をhtml のname 属性の値と合わせるだけで、パラメータを受け取ることができます。**login_input.html では、「id」と「password」のデータが送られるので、引数名も、int id, String password に合わせます。

コードの読み方

13行目、14行目

```
System.out.println(id);
System.out.println(password);
```

本当にid とpassword のデータを取得できたのか、コンソールに出力することで、検証をしています。結果は、以下の画像のように、入力した値を問題なく取得できているのがわかります。

コードの読み方

4行目

```
public String doLogin(UserForm userForm) {
```

doLogin メソッドの仮引数に、UserForm のオブジェクトを入れています。Spring では、**仮引数にForm クラスのオブジェクトを入れるだけで、リクエストパラメータがUserForm オブジェクトの対応するフィールド(id, password)に自動的にセットされます。**
よって、各リクエストパラメータを個別に取得するためのコードを書く必要がなくなります。

7～8行目

```
System.out.println(userForm.getId());
System.out.println(userForm.getPassword());
```

UserForm クラスに予め作成しておいた、getId()とgetPassword()メソッドを呼び出すことで、userForm オブジェクトにバインドされたID とパスワードを取得し、コンソールに出力します。

コードの読み方

13行目

```
public String doLogin(UserForm userForm, Model model) {
```

UserForm オブジェクトは、フォームから送信されたデータを受け取り、
リクエストスコープのオブジェクト(データ一時保管用の箱)を生み出すため、Model オブジェクトをメソッドの仮引数に入れます。

UserForm の時と同様に、**Spring ではメソッドの仮引数にオブジェクト名を書くと、オブジェクトを生成しなくても、使うことができます。**

コードの読み方

14～15行目

```
//リクエストスコープに、userForm のデータを格納
model.addAttribute("user", userForm);
```

この行は、UserForm オブジェクトを"user"というキーでモデルに追加しています。
第1 引数:キー名
第2 引数:格納したい変数

これにより、return で呼び出すlogin_success.html でも、userForm オブジェクトのデータが使用可能になります。

コードの読み方

17行目

```
return "login_success";
```

モデルに格納されたデータと共に、login_success.html を呼び出します。

このようにModel オブジェクトを使うと、Controller からView へデータを「一時的に入れておく箱」としてリクエストスコープを利用することができ、Spring のMVC パターンに沿った方法でデータを渡すことができます。

コードの読み方

2行目

```
<html xmlns:th="http://www.thymeleaf.org">
```

10 行と、11 行には、th:text というthymeleaf 固有の属性が登場していますが、そのためには、予め宣言をする必要があります。

10行目

```
社員番号 ：<span th:text = "${user.id}"></span><br>
```

この部分はThymeleaf 独自のスクレプト文法の一つの、th:text 属性を使用しています。

Controller からModel に格納した「user」というキーの後ろに、「.フィールド名」を記載することで、id の値を出力できます。
例えば、user.id が1234 であれば、出力は1234となります。

コードの読み方

11行目

```
パスワード：<span th:text = "${user.password}"></span>
```

ここでもth:text属性を使い、userオブジェクトのpasswordフィールドの値を表示しています。変数式を使用することで、動的に内容をWeb ページに埋め込むことができます。

コードの読み方

5～6行目

```
   @Autowired
private ProductRepository prodRepo;
```

全件検索をするためには、ProductRepository から利用できる「findAll」メソッドを呼び出さないといけません。**ProductRepository のメソッドを簡単に利用できるように、ProductRepository のフィールドを宣言して、@Autowired で依存性の注入を**しておきましょう。
この2 行のコードを書いておけば、[prodRepo]を使って、findAll メソッドを呼び出せます。

10行目

```
List<Product> list = prodRepo.findAll();
```

findAll メソッドを実行すると、Product テーブル内の全データを、List<Product>の形で取得できます。**List のImport 先は「java.util」ですので、ご注意ください。**

コードの読み方

12～13行目

```
model.addAttribute("products", list);
return "product/list";
```

最後は、データの塊である、list をView へ出力させるために、model に格納します。
このリストは、product が複数入っているので、キーは[products]にしました。
遷移先は、product フォルダの、list.html を指しています。

コードの読み方

8行目

```
<tr th:each="product: ${products}">
```

th:each は、**Java の拡張for 文と似た使い方**をします。指定されたコレクション（ここではproducts リスト）の各要素に対して、<tr> タグ（テーブルの行）を繰り返し生成します。

今回は、全部で9 個のデータがproducts リストの中に入っているので、products リストの要素が product 変数に9 回代入されます。**:を中心にして、右はController のmodel で格納したキー名、左はリストの中に格納されているものを意味する変数名を使います。**

今回は、リストの中に、複数のproduct インスタンスが格納されているため、「product」としました。

コードの読み方

9〜11行目

```
<td th:text="${product.id}"></td>
<td th:text="${product.name}"></td>
<td th:text="${product.price}"></td>
```

product 変数に1 行〜9 行までのデータが順番に格納されていますので、「product.フィールド名」でデータを出力することができます。つまり、各product オブジェクトの id、name、price を<td>の中に9 回出力するソースコードが生成されます。

コードの読み方

4行目

```
public String showProductById(int id, Model model) {
```

View から送信される主キーのname 属性が「id」ですので、引数名を一致させることで、リクエストパラメータを取得します。

コードの読み方

5行目

```
Product product = prodRepo.findById(id).orElse(null);
```

findById メソッドは、findAll と同じく、継承したJpaRepository の親に定義されているメソッドです。**引数には主キーを入れて、後ろに.orElse(null)を付けることで、主キー検索による、1 行の結果を取得**することができます。
.findById メソッドの戻り値は「Optional」という特殊な戻り値ですので、.orElse(null)をつけないとデータを取得できません。ご注意ください。

※.orElse(null)の代わりに、.get() というメソッドもありますが、検索結果がない場合は null ではなく、例外を出してアプリケーションを強制終了されてしまうので、お勧めしません。

findAll は、全件検索だったため、複数行が取得されることの前提でした。よって List<Product>型で受け取りました。
主キー検索は、必ず1 行のみ取得できるので、List での取得よりは、エンティティ単位での[Product product]で受け取るのが一般的です。

6行目

```
System.out.println(product);
```

検索する商品番号に「1」を入れて実行させてみると、以下のように1 行のデータを取得できているのがわかります。

コードの読み方

7行目

```
model.addAttribute("product", product);
```

取得した1 行のデータは、Product エンティティの中に格納されているので、Viewに出力させるため、model に格納します。

コードの読み方

5行目

```
List<Product> list = prodRepo.findByPrice(price);
```

5 行では、リクエストパラメータprice の値を取得し、prodRepo.findByPrice メソッドを実行しました。

6行目以降

```
System.out.println(list);
model.addAttribute("products", list);
return "product/list";
}
```

findAll メソッドを実行した時と、出力結果は同じくList になるメソッドですので、6 行以降はshowProductList メソッドと全く同じです。

コードの読み方

5行目

```
List<Product> list = prodRepo.findByNameAndPrice(name, price);
```

5 行では、リクエストパラメータname とprice の値を取得し、prodRepo.findByNameAndPrice メソッドを実行しました。

もし、入力値が更に増えるなら、Form クラスを使ってパラメータ取得しても良いでしょう。

6行目以降

```
System.out.println(list);
model.addAttribute("products", list);
return "product/list";
}
```

6 行以降はshowProductList メソッドと全く同じです。

コードの読み方

5行目

```
@Data
```

5 行の@Data ですが、エンティティの定義の際にご説明しました通り、@Data アノテーションをつけると、Getter&Setter メソッドは勿論、equals やtoString、hashCode メソッドが自動生成されますので、とても便利です。

コードの読み方

4行目

```
public String insertProduct(ProductForm productForm, Model model) {
```

ユーザからの複数の入力値を受け取るために、productForm クラスをメソッドの引数に入れます。リクエストデータは、Form クラスのフィールドに自動でセットされます。

5行目

```
Product product = new Product();
```

商品の登録をするためには、**予め商品1 個分をを表す、エンティティのインスタンスを生成する必要があります**ので、new でProduct のオブジェクトを生成します。

コードの読み方

8行目

```
product = prodRepo.save(product);
```

先に説明します。Insert のSQL 文を実行させるためには、findAll と同じく、既にリポジトリに定義されている、[save]メソッドを実行すれば、登録処理が完了します。

しかし、エンティティのインスタンスであるproduct は、5 行でオブジェクト生成はしたものの、データを入れないと、空っぽの状態のままになってしまいます。ユーザからのデータを受け取っているのは、product ではなく、4 行のProductForm の方ですので、データをコピーする必要があります。

コードの読み方

6〜7行目

```
product.setName(productForm.getName());
product.setPrice(productForm.getPrice());
```

save メソッドの実行前に、Form インスタンス内のデータを、product へコピーさせます。

この作業により、空っぽだったproduct にも、ユーザからのデータが格納されました。

8〜9行目

```
product = prodRepo.save(product);
            model.addAttribute("product", product);
```

戻り値としては、登録完了した商品情報をエンティティのインスタンスとして取得できます。ここには、自動割り当てされたid も含まれます。結果画面へ表示するため、model に格納します。

10行目

```
return "/product/result";
```

本当に登録が完了したのか確認するために、商品データを出力する結果画面を表示させます。

コードの読み方

9行目

```
<td th:text="${product.id}"></td>
```

9 行目のproduct.id に注目してください。save メソッドの戻り値には、自動採番により割り当てされたID 番号も含まれているので、表示できます。

コードの読み方

3行目

```
@GetMapping("update/{id}")
```

View から動的URL が送られる場合、同じく動的パラメータを指定する必要があります。書き方は、{ パラメータ名 } で記載します。

4行目

```
public String showUpdateInputView(@PathVariable int id, Model model) {
```

3 行で指定した動的URLの文字列と一致する引数名に**@PathVariable アノテーションをつけると、動的URL の値を取得**することができます。動的URL は、{id}なので、int id で合わせます。

5行目から

```
 Product product = prodRepo.findById(id).orElse(null);
 model.addAttribute("product", product);
 return "product/update_input";
}
```

5 行からは、更新対象の既存の値をfindById メソッドで取得+Model に格納し、update_input.html へ遷移させています。

コードの読み方

4行目

```
product.setId(productForm.getId());
```

登録の際は、主キーである「id」をProductエンティティ内にセットしていませんでしたが、更新処理の際は、必ず主キーのフィールドをセットさせます。

7行目

```
product = prodRepo.save(product);
```

save メソッドの引数に入れたエンティティのフィールドに、主キー(id)の値があったら、新しい値に上書きされる更新処理が実行されます。

コードの読み方

5行目

```
@Service
```

サービス実装クラスには、@Service アノテーションを付けます。忘れがちですので、注意してください。

6行目

```
@RequiredArgsConstructor
```

3.3.1「依存性の注入」の【依存性の注入方法③】でご紹介した書き方です。
9-10 行目のコードと連携します。
単体テストのしやすさのために、サービス実装クラスではこの方法が推奨されます。

7行目

```
public class ProductServiceImpl implements ProductService{
```

前の手順で作成したサービスインターフェースと繋げるために、implements します。

9～10行目

```
//@Autowired は不要
private final ProductRepository prodRepo;
```

DB 処理のためProductRepository を利用したいので、依存性の注入を行います。
6 行目と合わせて記載することで、コンストラクタを自動生成できます。

13～14行目

```
public Product findProductById(int id) {
return prodRepo.findById(id).orElse(null);
```

Controller に記載していたDB 処理部分だけ分離させ、DB から取得した商品データをそのままreturn しています。ProductService インターフェースで定義した抽象メソッドと同じ呼び方をするように、オーバーライドする必要があります。
(リモコンのボタンを押したら、このメソッドの処理が行われるイメージ)

コードの読み方

2行目

```
@RequestMapping("product2")
```

Controller が変わったので、異なる基本URL を設定します。

7行目

```
private ProductService prodService;
```

DB処理は、ProductServiceの方で担当するので、このクラスで利用できるように、依存性の注入を行います。既存のProductRepository の依存性の注入は不要になります。

13行目

```
Product product = prodService.findProductById(id);
```

ここが一番大事です。今までは、Controller のメソッドの中で直接「prodRepo.メソッド」を実行する書き方でしたが、今回はServiceクラスに定義されているメソッドを実行させて、その戻り値を利用する書き方になりました。

コードの読み方

12行目

```
@Column(name = "emp_name", length = 30, nullable = false)
```

User テーブルを利用しますので、User エンティティも新しく定義する必要があります。

エンティティの定義については、前の章のProduct とほぼ同じですが、12 行のname 属性を見てください。テーブル内の列名は[emp_name]ですが、Java では通常、変数名の単語ごとの区切りに「アンダースコア(_)」を入れるよりは、大文字と小文字の組み合わせにする、「キャーメルケース」を使います。

よって、テーブル内の列名と変数名に違いが発生したら、明示的にname 属性をつける必要があります。

コードの読み方

3行目

```
User user = userRepo.findByEmpIdAndPassword(userForm.getEmpId(),
                    userForm.getPassword());
```

UserRepository に予め定義しておいたメソッドを実装しました。login_input.html からのリクエストパラメータは、userForm 内に自動格納されているため、getメソッドで取得し、引数に入れました。**検索結果は、最大1 件ですので、User エンティティで受け取ります。**

5行目

```
if (user == null) { //結果がnull なら、再度ログイン画面へ
```

社員ID とパスワードが一致したら、社員データを取得できますが、入力に誤りがあると、何も取得できないので、user 変数は、null のままになります。
このif 文は、**取得できなかった際は、もう一回ログイン画面へ遷移させることで、再入力を促す**コードです。

5 行のif 文内でreturn されると、即遷移するため、7 行以降が実行されることはありません。

8行目

```
return "main_page";
```

データを取得できたなら、ログイン成功ですので、main_page.html へ遷移させます。

main_page.html は、下記のように、簡単なhtml ファイルです。

コードの読み方

7行目

```
public String loginSession(UserForm userForm, HttpSession session) {
```

セッションは、HttpSession のインスタンスを利用することで、データをセッションスコープに格納できたり、セッション情報を消したりできます。**Spring のController にURL がマッピングされたメソッドのことを、改めて「ハンドラメソッド」といいます。このハンドラメソッドの引数にHttpSession を指定するだけで、裏でフレームワークが HttpSession のインスタンス生成と、セッション情報の取得を自動で行ってくれます。**

13行目

```
session.setAttribute("userData", user);
```

session.setAttribute メソッドは、model.addAttribute と同じような書き方をします。第一引数には、キーワードを記載して、第二引数には、データを指定します。このコードが実行されると、セッションスコープの中に、"userData"という名前でログインユーザの全情報を保持できるようになります。このデータは、**画面が変わっても、システム全域で呼び出すことができます。**

コードの読み方

3〜5行目

```
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.NotNull;
import jakarta.validation.constraints.Size;
```

Validationのバージョンによりますが、最新のValidation API は、jakarta.validation.constraintsパッケージの方を利用することが推奨されています。

11行目

```
@NotNull(message = "社員番号を入力してください")
```

このアノテーションを付与したフィールドに、**入力値がない場合は、入力エラーだと検知**してくれます。**Integerのフィールドは、**@NotNullを付与します。**message属性は、出力するメッセージをカスタマイズ**することができます。

14〜15行目

```
@NotBlank(message = "パスワードを入力してください")
@Size(max = 15, message = "15 文字以内で入力してください")
```

@NotNull と似ていますが、こちらは**文字列の入力有無チェックに特化**されています。未入力はもちろん、**空文字(“”)まで検知**することができます。
@Size は、**文字列の文字数制限**を設定できます。Max で最大文字数、min で最小文字数の設定ができます。

コードの読み方

3行目

```
public String loginValidation(@Valid UserForm userForm, BindingResult
bdResult) {
```

@Valid アノテーションは、対象のオブジェクト（この場合は UserForm）が持つValidation アノテーション（例えば @Size、@NotNull など）に基づいて**入力チェックを実行するために使用**されます。

3行目

```
public String loginValidation(@Valid UserForm userForm, BindingResult
bdResult) {
```

BindingResult は、@Valid アノテーションによる入力チェックの結果を保持するオブジェクトです。入力チェックエラーの詳細を取得するために使用されます。通常、メソッドの引数として渡されます。**必ず、@Valid をつけた直後（ここでは、第2 引数）に記載しないといけないルールがあります**ので、ご注意ください。

コードの読み方

4行目

```
if (bdResult.hasErrors()) {
```

BindingResult の **hasErrors メソッドは、入力チェック中にエラーが発生したかどうかを確認するために使用されます。エラーが存在する場合は true を、そうでない場合は false を返します。**このメソッドを利用して条件分岐を行い、エラーがある場合は入力フォームに戻り、エラーメッセージをユーザに表示します。

コードの読み方

4行目

```
<form th:action="@{/login4}" method="post" th:object="${userForm}">
```

th:object 属性は、**「選択対象オブジェクト」**と呼ばれ、フォームのデータをController のオブジェクト（この場合は UserForm オブジェクト）にバインド（結びつけ）するために使用されます。

formタグに記述することで、フォームの入力フィールドの値が、UserForm オブジェクトの対応するプロパティに自動的に格納されます。
つまり、Controller のメソッド内でModel にデータを格納しなくても、Form タグの中で、Thymeleaf 上でデータを受け取れるということです。

フォーム内では、*{...}という選択式を使用して、userForm オブジェクトのプロパティに簡潔にアクセスできます。**userForm.empId ではなく、*{empId}だけでアクセスできるようになります。**

コードの読み方

6行目

```
<p>社員番号　：<input type = "text" th:field="*{empId}"/>
```

th:field 属性は、対応するフィールドの name, id, および value 属性を自動的に生成します。ですので、name、id、value を省略できます。

自動生成されるvalue 属性により、フォーム送信後に入力チェックエラーが発生した場合でも、**ユーザが入力した値がフィールドに保持されます**。これにより、ユーザは入力値を再度入力する手間を省くことができます。

なお、*{empId}は、userForm オブジェクトのempId プロパティを参照しています。

7行目

```
<span th:errors="*{empId}"></span></p>
```

このコードは、empId フィールドに関連する入力チェックエラーがある場合に**エラーメッセージを表示**するために使われます。エラーがない場合は、何も表示されません。

フォームを送信したとき、もし empId フィールドの値が入力チェックのルールに違反していれば（例えば、必須フィールドが空である、など）、 タグの中に入力チェックごとのエラーメッセージが表示されます。

コードの読み方

2行目

```
<html xmlns:th="http://www.thymeleaf.org">
```

Thymeleaf テンプレートで使われる HTML の宣言です。この宣言により、Thymeleaf のルールが HTML ページに追加されます。それによって、Thymeleaf の特有の属性や構文が HTML タグ内で認識され、正しく処理されます。

6行目

```
<meta name="viewport" content="width=device-width, initial-scale=1.0"
/>
```

この宣言は、Thymeleaf の構文ではありませんが、Web ページがモバイルデバイスで表示される際のビューポート（表示領域）の設定に使われます。デバイスがどのような画面サイズを持っていても、Web ページはそれに適応するように表示されます。

コードの読み方

3行目

```
xmlns:layout="http://www.ultraq.net.nz/thymeleaf/layout">
```

レイアウトファイルを定義するためには、既存のhtml 属性に、xmlns:layout 属性を追記する必要があります。html 要素を2 回書いたわけではなく、html 要素に2 つの属性を追加していることに気をつけてください。

8行目

```
<link rel="stylesheet" th:href="@{/css/layout.css}" />
```

レイアウト専用のcss ファイルを別途作成し、読み込ませました。css ファイルの詳細は、デザインの領域ですので、章末の【参考】の部分をご覧ください。

27～29行目

```
<main layout:fragment="main_content">
 <!-- 各ページ固有のコンテンツがここに入ります -->
</main>
```

この部分のコードは、**中身がありません**。その理由は、ここは**メインコンテンツが配置される場所**だからです。
「layout:fragment=“識別名”」の属性をつけると、この部分に別のhtml ファイル(コンテンツ)のfragment 識別名が一致しているコードがコピーされ、**置き換えられます**。

コードの読み方

3行目

```
xmlns:layout="http://www.ultraq.net.nz/thymeleaf/layout"
```

レイアウトにコピーするコンテンツ側のHTML にも、同じくレイアウト関連のルールを宣言しておきます。

4行目

```
layout:decorate="~{/layout}">
```

<html>の属性の中にlayout:decorate 属性も追記します。この部分の属性値は、layout.htmlの場所をURL で指定すればOK です。

例えば、layout がproduct フォルダ内にあるなら、”~{/product/layout}” のように記述すればOK です。URL を間違えると、レイアウト機能が使えませんので、ご注意ください。

9～13行目

```
<main layout:fragment="main_content">

<!--既存のコンテンツのソースコード-->

</main>
```

既存の<body>内に書いていたソースコードを、全てこの要素で囲みます。先ほどのlayout.html のfragment 識別名が“main_content”でしたので、同じ値を入れます。そうすることで、9-13 行内の全てのHTML コードが、layout.html の28 行にコピーされ、レイアウト+コンテンツの新しいHTML が生成されます。insert.html へreturn して一覧画面を開くと、自動でlayout.html の共通部分まで生成された画面が表示されます。

コードの読み方

1行目

```
<div class="container">
```

今回のサンプルでは、外側を container でコンテナを作っています。画面の中央にコンテンツが配置されます。もし、container-fluid にすると、横幅いっぱいに変わります。

コードの読み方

2行目

```
<div class="row">
```

コンテナの中に、row をclass 属性として持つdiv 要素を入れます。後述のカラムを配置するためにも、必ずrow が必要です。まるでtable 要素に、<tr>がないと、<td>も書けないのと同じです。

3〜5行目

```
<div class="col-6 bg-primary-subtle">Column 1</div>
<div class="col-3 bg-danger-subtle">Column 2</div>
<div class="col-3 bg-success-subtle">Column 3</div>
```

1 つのrow の中に、複数のカラムを配置できます。このサンプル通りだと、以下のように表示されます。

コードの読み方

6〜7行目

```
    </div> <!-- END row1-->
</div><!-- END container-->
```

グリッドシステムでは、 要素を使うことが多いので、閉じタグだけでは、どの要素の閉じタグなのか、区別がつきません。そのため、**どの要素の閉じタグなのかを明確にするため、コメントを記載することが多いです。**特に、row が複数ある場合は「END row1」や、「END row2」とかでコメントを残すことで実際の範囲も読みやすくなるので、重要です。

コードの読み方

5行目

```
<div class="col-md-3"> <!-- START sidebar -->
```

フォームとは直接関係がないコードですが、横幅の3 割のみ占めるように、限定をさせることで、左サイドバーのようなレイアウトを作ることができます。検索バーは、サイドバーに配置させることが多いので、予めカラムを生成しました。

8行目

```
<form class="row">
```

〈div〉ではなく、〈form〉要素をrow として生成すると、フォームを別途の行として扱えることができます。

9行目・13行目

```
<div class="col-8">
<div class="col-4">
```

入力フォームに8、ボタンに4 の割合でカラムを生成しています。

10行目

```
<input type="text" class="form-control"
```

コードの読み方

10行目

```
<input type="text" class="form-control"
```

Bootstrap のフォーム関連クラスの中で、一番大事なクラスです。テキストフォームにこのクラスを付与することで、**Bootstrap 式のテキストフォームのデザインを適用**することができます。